

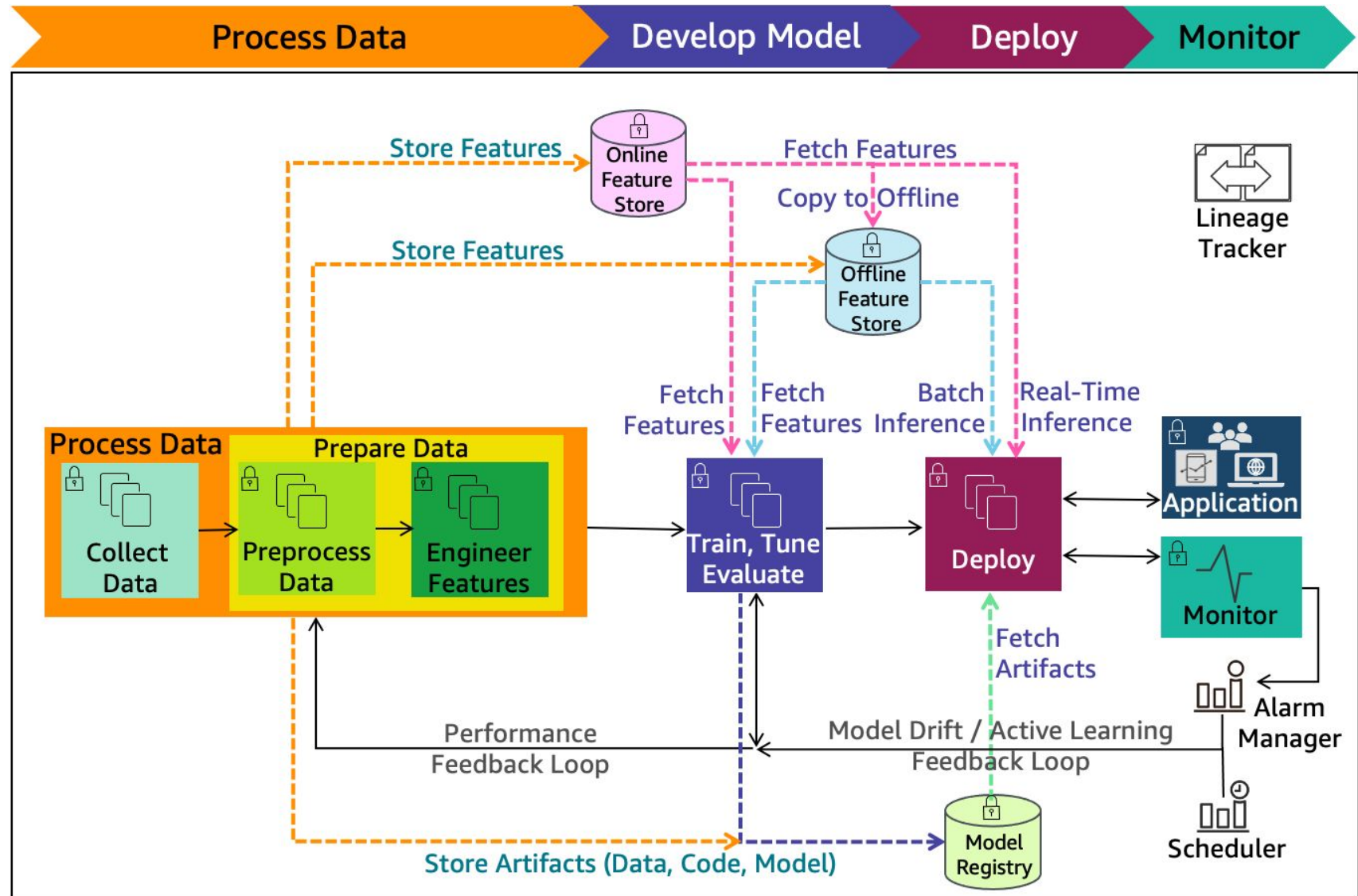
CS611 Machine Learning Engineering

Lecture 3: Data Preparation

Ulysses CHONG Min Zhen

ulysseschong@smu.edu.sg

Machine Learning Lifecycle



Outline

Data Preparation

- Medallion Architecture

- Data cleaning and preprocessing

- Splitting data

- Feature engineering

- Feature store

Big Data Processing with PySpark

Lab

- Data preprocessing with PySpark

- Group Project + Assignment 1

Class Case Study Discussion

Case Study

You are the Data Scientist at a bank.

You are tasked to build a credit scoring model to predict if a customer will default on a loan.

At the moment, you have access to the loan management system which shows daily repayments of loans by customers.

What shall we build?

Case Study (data available)

Loan Management System (LMS)

SELECT * FROM lms.loan_daily LIMIT 100

loan_id	customer_id	loan_amt	due_amt	paid_amt	overdue_amt	balance_amt	snapshot_date
1	123	1000	100	100	0	700	2024-08-01
1	123	1000	100	100	0	800	2024-07-01
1	123	1000	100	100	0	900	2024-06-01
2	234	1000	100	0	300	900	2024-08-01
2	234	1000	100	0	200	900	2024-07-01
2	234	1000	100	0	100	900	2024-06-01
2	234	1000	100	100	0	900	2024-05-01

Case Study (data available)

Loan Management System (LMS)

```
SELECT * FROM lms.loan_daily LIMIT 100
```

loan_id	customer_id	loan_amt	due_amt	paid_amt	overdue_amt	balance_amt	snapshot_date
3	345	1000	100	100	0	300	2024-08-01
3	345	1000	100	350	0	400	2024-07-01
3	345	1000	100	0	250	750	2024-06-01
3	345	1000	100	100	150	750	2024-05-01
3	345	1000	100	50	150	850	2024-04-01
3	345	1000	100	0	100	900	2024-03-01
3	345	1000	100	100	0	900	2024-02-01

Case Study

You are the Data Scientist at a bank.

You are tasked to build a credit scoring model to predict if a customer will default on a loan.

At the moment, you have access to the loan management system which shows daily repayments of loans by customers.

Activity 1: Label engineering

Align modeling approach (regression? classification? smth else?)

Align label definition

Write pseudo-code to engineer label (medallion architecture compliant)

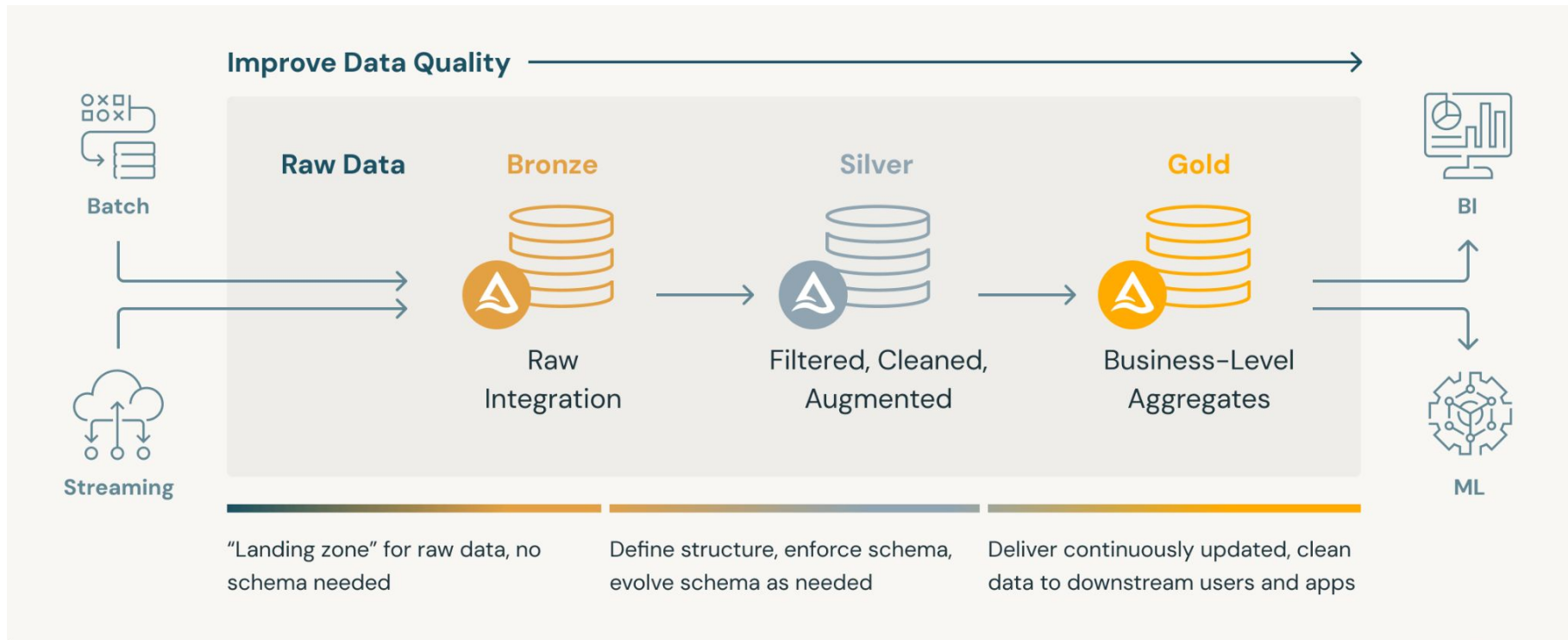
Activity 2: Feature engineering

Identify and design 1 feature

Write pseudo-code to engineer feature (medallion architecture compliant)

Data Preparation

Organizing data with Medallion Architecture



Read here for bronze / silver / gold guidance: <https://www.databricks.com/glossary/medallion-architecture>

Data Processing

Data Processing

Data Collection

Data Preparation (Wrangling during Interactive Data Analysis) Leverage **Visualization** for Exploratory Data Analysis (EDA)

Data Collection

Label

Ingest (Streaming, Batch)

Aggregate

Data Preprocessing

Clean (Replace, Impute,
Remove Outliers, Duplicates)

Partition (Train, Validate, Test)

Scale(Normalize, Standardize)

Unbias, Balance
(Detection & Mitigation)

Augment

Feature Engineering

Feature Selection

Feature Transformation

Feature Creation
(Encoding, Binning)

Feature Extraction
(Automated in Deep Learning)

Data Preparation

- Raw data are often not compatible with algorithm input format
- Even worse, real world dataset are generally very noisy
 - For example: inconsistent unit of measures, inconsistent values representing same meaning, wrong values, missing values, etc.
- Expect your input to be the same as the one in your deployment environment

Data Preparation

- We generally employ several exploratory data analysis techniques such as descriptive statistics, correlation analysis, data visualization
- Popular open source python libraries for data preparation:
 - pandas (https://pandas.pydata.org/docs/user_guide/index.html)
 - numpy (<https://numpy.org/doc/stable/user/index.html>)
 - scikit-learn (<https://scikit-learn.org/stable/tutorial/index.html>)

Data Cleaning

- Several data cleaning tasks:
 - Simple rules to handle inconsistency in data
 - Duplicates
 - Missing values: remove or impute
 - Outlier: remove or impute
- Take note of the risk of removing data instances / features

Splitting Data

- We do not use all data in our dataset for machine learning training
- Split dataset into training, validation, and testing set
 - Common splitting: 80% training, 10% validation, and 10% testing. Ensure you have sufficient testing set.
 - For imbalance datasets, maintain the same class distribution on all sets. Use stratified splitting.
- For very small dataset, people may use cross validation
- Be careful of data leakage
 - It happens when your model gets data during inference that it should not have access to.

Detecting and Mitigating Bias

- Biases are imbalances in the accuracy of prediction across different groups such as age or income bracket.
 - <https://aws.amazon.com/blogs/machine-learning/learn-how-amazon-sagemaker-clarify-helps-detect-bias/>
- Measuring data bias:
<https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-measure-data-bias.html>

Feature Engineering

- Feature creation
 - Example: one-hot encoding, binning, calculated features
- Feature selection
 - Example: selecting subset of relevant features
- Feature transformation
 - Example: log transform, creating domain specific features
- Feature extraction
 - Example: PCA, LDA, bag of words, n-gram

Feature Store

- Centralized store for features and associated metadata so that features can be easily discovered and reused.
 - Online feature store: for low latency real-time inference use cases
 - Offline feature store: for training and batch inference
- Features are stored in a collection called a feature group.
- Metadata may include short description, storage configuration, event time, and tags to store additional information such as the author, data source, version, etc.
- Feature group names and associated metadata should not contain any personal identifiable information (PII) or confidential information.

15 mins break

**.. and then you'll change the way you
use pandas forever!!**

Big Data Processing with PySpark

PySpark; “Pandas’s buff sibling on steroids” - anon

PySpark



source: <https://www.youtube.com/watch?v=IELMSD2kdmk>

PySpark

“PySpark is the Python API for Apache Spark. It enables you to perform real-time, large-scale data processing in a distributed environment using Python. It also provides a PySpark shell for interactively analyzing your data.

PySpark combines Python’s learnability and ease of use with the power of Apache Spark to enable processing and analysis of data at any size for everyone familiar with Python.

PySpark supports all of Spark’s features such as Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib) and Spark Core.”

source: <https://spark.apache.org/docs/latest/api/python/index.html>

PySpark

Spark SQL and
DataFrames

Pandas API on
Spark

Structured
Streaming

Machine
Learning
MLlib

Spark Core and RDDs

source: <https://spark.apache.org/docs/latest/api/python/index.html>

PySpark

Pandas ↔ Polars ↔ SQL ↔ PySpark

 [blog.DailyDoseofDS.com](https://blog.dailydoseofds.com)

Operation	Pandas	Polars	SQL	PySpark
Import	<code>import pandas as pd</code>	<code>import polars as pl</code>	-	<code>from pyspark.sql import SparkSession spark = SparkSession.builder.appName("ABCD")</code>
Read CSV	<code>df = pd.read_csv(file)</code>	<code>df = pl.read_csv(file)</code>	<code>LOAD DATA INFILE 'data.csv' INTO TABLE table FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n' IGNORE 1 ROWS;</code>	<code>df = spark.read .csv("data.csv")</code>
Print first 10 (or k) rows	<code>df.head(10)</code>	<code>df.head(10)</code>	<code>SELECT * FROM table LIMIT 10;</code>	<code>df.show(10)</code>
Dimensions	<code>df.shape</code>	<code>df.shape</code>	<code>SELECT count(*) FROM table;</code>	<code>df.count()</code>
			<code>SELECT count(*) FROM INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'table';</code>	<code>len(df.columns)</code>
Datatype	<code>df.dtypes</code>	<code>df.dtypes</code>	<code>DESCRIBE table;</code>	<code>df.printSchema()</code>
Select column(s)	<code>df[["col1", "col2"]]</code>	<code>df[["col1", "col2"]]</code>	<code>SELECT column FROM table;</code>	<code>df.select("col1", "col2")</code>
Filter Data	<code>df[df.column > 10]</code>	<code>df[df.column > 10]</code>	<code>SELECT * FROM table where column>10;</code>	<code>df.filter(df["column"]>10)</code>
		<code>df.filter(pl.col("column") > 10)</code>		
Sort	<code>df.sort_values("column")</code>	<code>df.sort("column")</code>	<code>SELECT * FROM table ORDER BY column;</code>	<code>df.orderBy("column")</code>
Fill NaN	<code>df.column.fillna(0)</code>	<code>df.column.fill_nan(0)</code>	<code>UPDATE table SET column=0 WHERE column IS NULL;</code>	<code>df.na.fill(0)</code>
Join	<code>pd.merge(df1, df2, on="col", how="inner")</code>	<code>df1.join(df2, on="col", how="inner")</code>	<code>SELECT * FROM table1 JOIN table2 ON (table1.col = table2.col);</code>	<code>df1.join(df2, on="col", how="inner")</code>
Concatenate	<code>pd.concat((df1, df2))</code>	<code>pl.concat((df1, df2))</code>	<code>SELECT * FROM table1 UNION ALL table2;</code>	<code>df1.union(df2)</code>
Group	<code>df.groupby("column"). agg_col.mean()</code>	<code>df.groupby("column"). agg(pl.mean("agg_col"))</code>	<code>SELECT column, avg(agg_col) FROM table GROUP BY column;</code>	<code>df.groupBy("column"). agg(avg("agg_col"))</code>
Unique values	<code>df.column.unique()</code>	<code>df.column.unique()</code>	<code>SELECT DISTINCT column FROM table;</code>	<code>df.select("column"). distinct()</code>
Rename column	<code>df.rename(columns = {"old_name": "new_name"})</code>	<code>df.rename(mapping = {"old_name": "new_name"})</code>	<code>ALTER TABLE table RENAME COLUMN old_name TO new_name;</code>	<code>df.withColumnsRenamed({"old_name": "new_name"})</code>
Delete column	<code>df.drop(columns = ["column"])</code>	<code>df.drop(name = ["column"])</code>	<code>ALTER TABLE table DROP COLUMN column;</code>	<code>df.drop("col1", "col2")</code>

source:

<https://blog.dailydoseofds.com/p/15-pandas-polars-sql-pyspark-translations>

Lab

Lab Activities

1. Set up lab 2 git folder
2. Load Bronze table with PySpark and process to Silver table
3. Save Silver table as parquet file
4. Build Gold table with PySpark for engineered labels
5. Save Gold table as parquet file
6. (Bonus) convert to py executable scripts

Dataset to be provided by instructor

original source datasets for instructors: <https://www.kaggle.com/datasets/parisrohan/credit-score-classification>

Homework

Assignment 1 released!

Group Project

Form groups of 5 people

Start working towards your proposal presentation Week 6

An aerial night photograph of Singapore. The foreground shows a large, modern building with a brown roof and glass facade, identified by the 'SMU' logo. The background features a dense city skyline with numerous illuminated skyscrapers under a dark sky. The text 'Questions?' is overlaid in a bold, orange font on the left side of the image.

Questions?

Thank You